

Java Programming Training

Day 3

Complete Program Reference
8 Programs · Full Theory · Explanations · Trace Tables

Section	Topic	Programs
§ A	Do-While Loop	01 DoWhileGuessNumber.java 02 Game.java
§ B	Functions (Modular)	03 PassByValueDemo.java 04 ModularApp.java
§ C	Arrays & Calculations	05 MarksPercentageGradeCal.java 06 GradeFunction approach
§ D	Number Logic	07 ArmstrongNumbers1To1000.java 08 PrimeinRange.java
§ E	Master Quick Reference	All 8 programs · Concept cheat sheet

§ A

Do-While Loop

Guaranteed first execution — condition checked AFTER the loop body runs

■ Theory — What is a do-while Loop?

A **do-while loop** guarantees the loop body executes **at least once**, regardless of the condition. The condition is evaluated only *after* the body runs — the opposite of a while loop. This makes it ideal whenever the first execution is unconditional, such as showing a menu, prompting a user for input, or making at least one guess in a game.

Difference Between while and do-while

while loop	do-while loop
Condition checked before execution	Condition checked after execution
May execute 0 times	Executes at least 1 time
Syntax: while (cond) { ... }	Syntax: do { ... } while (cond);

■ Syntax (Java)

```
do {
    // code to execute — runs at LEAST ONCE
} while (condition);
```

PROGRAM 01

DoWhileGuessNumber.java

Classic number guessing game — do-while ensures at least one guess

■ CONCEPT — do-while for Guessing — Why Not a while Loop?

In a guessing game the user **must** guess at least once before the result can be checked. A while loop could theoretically skip the body if the condition is false from the start. A do-while **guarantees** the prompt appears, the input is read, and feedback is given before the condition is tested. This is the canonical use-case for do-while.

■ DoWhileGuessNumber.java

```
package Day3;

import java.util.Scanner;

public class DoWhileGuessNumber {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int secretNumber = 7;
        int guess;

        do {
            System.out.print("Guess the number: ");
            guess = sc.nextInt();

            if (guess > secretNumber) {
                System.out.println("Too High!");
            } else if (guess < secretNumber) {
                System.out.println("Too Low!");
            }
        } while (guess != secretNumber);

        System.out.println("Correct Guess!");
    }
}
```

```
}  
}
```

■ Output

```
Guess the number: 3  
Too Low!  
Guess the number: 9  
Too High!  
Guess the number: 7  
Correct Guess!
```

Iteration	guess entered	guess vs secretNumber (7)	Output
1	3	3 < 7	Too Low!
2	9	9 > 7	Too High!
3	7	7 == 7	Correct Guess! → loop exits

PROGRAM 02

Game.java

Player vs secret number — tracks total guesses with a counter

■ CONCEPT — Guess Counter — Tracking Attempts with a Variable

This extends the basic guessing game by adding a **count** variable that increments on every iteration of the do-while loop. After the loop exits (correct guess), the total number of attempts is printed. The loop body always runs once before checking **player1 != player2**, ensuring the first prompt and guess always happen.

■ Game.java

```
package Day3;  
  
import java.util.Scanner;  
  
public class Game {  
    public static void main(String[] args) {  
        int player1 = 30;  
        int player2 = 0;  
        int count = 0;  
        Scanner sc = new Scanner(System.in);  
  
        do {  
            count++;  
            System.out.println("Guess the number");  
            player2 = sc.nextInt();  
            if (player2 > player1) {  
                System.out.println("please guess a smaller no");  
            } else if (player2 < player1) {  
                System.out.println("please guess a bigger no");  
            } else {  
                System.out.println("Hadippaaa number guessed");  
            }  
        } while (player1 != player2);  
  
        System.out.println(count + " is the no of guess you have taken");  
    }  
}
```

■ Output

```
Guess the number
10
please guess a bigger no
Guess the number
40
please guess a smaller no
Guess the number
30
Hadippaaa number guessed
3 is the no of guess you have taken
```

Iteration (count)	player2	Condition	Output
1	10	10 < 30	please guess a bigger no
2	40	40 > 30	please guess a smaller no
3	30	30 == 30	Hadippaaa number guessed
Exit	—	player1 == player2 → false loop	3 is the no of guess...

■ TIP

count++ is the FIRST line inside the loop — it counts every attempt including the final correct one. If you placed count++ after the if-else, the last guess would still be counted, but putting it first is cleaner and ensures no guess is missed.

§ B

Functions (Modular Programming)

Pass by value, return values, static methods, and menu-driven modular apps

■ Theory — What are Functions in Java?

A **function (method)** is a named, reusable block of code that performs a specific task. Instead of writing the same logic repeatedly, you write it once in a method and **call** it whenever needed. Java uses **static** methods inside a class to allow calling without creating objects. The key concepts are: **parameters** (inputs), **return type** (what comes back), and **scope** (local variables only exist inside their method).

Part	Syntax Example	What It Does
Return type	static int totalMarks(...)	Declares what type the method sends back
Parameters	int s1, int s2, ...	Inputs the method receives (local copies)
Return stmt	return s1+s2+s3+s4+s5;	Sends the result back to the caller
void method	static void grade(double p)	Does work but returns nothing
Call	int total = totalMarks(sub1, sub2, ...)	Invokes the method

PROGRAM 03

PassByValueDemo.java

Demonstrates pass-by-value — original variable is never modified inside a method

■ CONCEPT — Pass By Value — Java's Default Parameter Passing

In Java, **primitive types** (int, double, etc.) are passed to methods **by value** — a *copy* of the variable is given to the method. Changes inside the method do NOT affect the original variable. The **parameter's scope is local** to the method — the inline comment in the source code says exactly this: *// parameter ka scope -> local*. The return value is the only way the modified value comes back out.

■ PassByValueDemo.java

```
package Day3;

public class PassByValueDemo {
    static int Increment(int num) { // parameter ka scope -> local
        num++;
        System.out.println(num);
        return num;
    }

    public static void main(String[] args) {
        int data = 10;

        System.out.println(data); // pass by value -> copy
        int result = Increment(data);
        System.out.println(result);
    }
}
```

■ Output

```
10
11
11
```

Step	Variable	Value	Where
Before call	data	10	main() — original
Inside Increment()	num	10 (copy of data)	local to Increment()
After num++	num	11	local — data unchanged
println(num)	—	prints 11	inside Increment()
return num	result	11 (returned value)	main()
println(result)	—	prints 11	main()
println(data)	data	still 10 (unchanged!)	main()

■ WHY THIS MATTERS

If Java used pass-by-reference, **data** would become 11 after the call. Because Java passes a **copy**, **data stays at 10**. The only way to get the updated value out is via **return**.

PROGRAM 04

ModularApp.java

Full banking app with static methods — Deposit, Withdraw, Balance, FD, while+switch

■ CONCEPT — Modular Banking App — Separating Concerns into Methods

This program upgrades the BankWhileLoop from Day 2 by extracting each operation into its own **static method**:

Deposit(), **Withdraw()**, **showbalance()**, and **FD()**. Each method has a single responsibility. The main() only handles the menu and routing (while+switch). Notice that **balance** is passed in and **returned** from Deposit/Withdraw — this is the pass-by-value pattern in action.

■ ModularApp.java

```
package Day3;

import java.util.Scanner;

public class ModularApp {

    static int Deposit(int balance, int amt) {
        balance += amt;
        System.out.println("hehehe ho gye paise deposit!!!");
        return balance;
    }

    static int Withdraw(int balance, int amt) {
        if (balance > amt) {
            balance -= amt;
        }
        System.out.println("hehehe nikal gye paise !!!");
        return balance;
    }

    static void showbalance(int balance) {
        System.out.println(balance);
    }

    static void FD(int balance, int time) {
        System.out.println("Your amt after " + time + " years will be "
            + (balance * 7 * time) / 100);
    }
}
```

```
public static void main(String[] args) {
    System.err.println("1.Withdrawl 2.Deposit 3.CurrentBalance 4.FD 5.exit");
    int balance = 2000;
    Scanner obj = new Scanner(System.in);
    int option = 0;
    while (option != 5) {
        System.out.println("Enter your operation!!");
        option = obj.nextInt();
        switch (option) {
            case 1:
                System.out.println("Enter the amt to withdraw");
                int amt = obj.nextInt();
                balance = Withdraw(balance, amt);
                break;
            case 2:
                System.out.println("Enter the amt to deposit");
                int amtToDeposit = obj.nextInt();
                balance = Deposit(balance, amtToDeposit);
                break;
            case 3:
                showbalance(balance);
                break;
            case 4:
                System.out.println("Enter the time to deposit your money");
                int time = obj.nextInt();
                FD(balance, time);
                break;
        }
    }
}
```

■ Output

```
1.Withdrawl 2.Deposit 3.CurrentBalance 4.FD 5.exit
Enter your operation!!
3
2000
Enter your operation!!
2
Enter the amt to deposit
1000
hehehe ho gye paise deposit!!!
Enter your operation!!
3
3000
Enter your operation!!
5
```

Method	Parameters	Returns	Key Operation
Deposit()	balance, amt	int (new balance)	balance += amt

Method	Parameters	Returns	Key Operation
Withdraw()	balance, amt	int (new balance)	if balance > amt: balance -= amt
showbalance()	balance	void	System.out.println(balance)
FD()	balance, time	void	(balance * 7 * time) / 100

■ **BUG NOTE — Withdraw Without Balance Check Message**

The current `Withdraw()` always prints 'nikal gye paise' even when `balance <= amt` (insufficient funds). A complete version would add an **else** branch: **`else { System.out.println("Insufficient balance"); }`**.

§ C

Arrays & Marks Calculator

Arrays, for-loops for input/output, percentage calculation, grade logic

■ Theory — What is an Array?

An **array** is a fixed-size collection of elements of the same type, stored in contiguous memory. In Java, **int[] marks = new int[5]** creates an array of 5 integers indexed 0 to 4. Arrays eliminate the need for separate variables (sub1, sub2, sub3...) when the data is uniform. A **for loop** is the natural companion — the loop variable serves as the array index.

Operation	Syntax	Example
Declare + create	<code>int[] arr = new int[5];</code>	Creates 5 slots: <code>arr[0]..arr[4]</code>
Assign element	<code>arr[i] = value;</code>	<code>arr[2] = 75;</code>
Access element	<code>arr[i]</code>	<code>System.out.println(arr[2]);</code> → 75
Loop over array	<code>for (int i=0; i<5; i++)</code>	Accesses each slot in order
Accumulate total	<code>total += arr[i];</code>	Adds each element to running sum

PROGRAM 05

MarksPercentageGradeCal.java

Array-based marks input — total, percentage, custom grade messages

■ CONCEPT — Array + Loop — The Standard Input-Accumulate-Display Pattern

This program uses an **int[] marks = new int[5]** array and a single **for loop** for both input and display, replacing five separate variables. The total is accumulated inside the input loop with **total += marks[i]**. The grade messages are intentionally informal/humorous — this is a real class exercise demonstrating the logic works correctly regardless of the message strings.

■ MarksPercentageGradeCal.java

```
package Day3;

import java.util.Scanner;

public class MarksPercentageGradeCal {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int[] marks = new int[5];
        int total = 0;
        int outOf = 500;

        // Input marks
        for (int i = 0; i < 5; i++) {
            System.out.print("Enter marks for Subject " + (i + 1) + ": ");
            marks[i] = sc.nextInt();
            total += marks[i];
        }

        // Calculate percentage
        double percentage = (total * 100.0) / outOf;

        // Grade calculation
        String grade;

        if (percentage >= 90) {
```

```

    grade = "A";
} else if (percentage >= 80) {
    grade = "B";
} else if (percentage >= 70) {
    grade = "C";
} else if (percentage >= 60) {
    grade = "D";
} else {
    grade = "F";
}

// Display result
System.out.println("\n----- RESULT -----");
for (int i = 0; i < 5; i++) {
    System.out.println("Subject " + (i + 1) + ": " + marks[i]);
}

System.out.println("Total Marks: " + total);
System.out.println("Out Of: " + outOf);
System.out.println("Percentage: " + percentage + "%");
System.out.println("Grade: " + grade);

sc.close();
}
}

```

■ Output

```

Enter marks for Subject 1: 85
Enter marks for Subject 2: 90
Enter marks for Subject 3: 75
Enter marks for Subject 4: 88
Enter marks for Subject 5: 80
----- RESULT -----
Subject 1: 85
Subject 2: 90
Subject 3: 75
Subject 4: 88
Subject 5: 80
Total Marks: 418
Out Of: 500
Percentage: 83.6%
Grade: B

```

i	marks[i] entered	total (after +=)	Action
0	85	85	marks[0]=85
1	90	175	marks[1]=90
2	75	250	marks[2]=75
3	88	338	marks[3]=88
4	80	418	marks[4]=80
—	Loop ends	—	percentage = (418*100.0)/500 = 83.6% → Grade: B

■ TIP

Why (total * 100.0)? Writing **100.0** instead of **100** forces Java to treat the multiplication as a **double** operation. Without the .0, integer division would truncate 83.6 → 83, losing the decimal part entirely.

**PROGRAM
06**

GradeFunction (Modular Approach)

Same grade logic refactored into three static methods — totalMarks(), percentage(), grade()

■ CONCEPT — Modular Grade Calculator — Single-Responsibility Functions

The notes introduce a **GradeFunction** version that splits the grade calculator into three static methods. **totalMarks()** returns an int, **percentage()** returns a double, and **grade()** is void (only prints). This is the **Single Responsibility Principle** in action — each function does exactly one thing. **main()** only reads input and calls functions.

■ GradeFunction.java

```
package Day3;

import java.util.Scanner;

public class GradeFunction {
    // Function to calculate total
    static int totalMarks(int s1, int s2, int s3, int s4, int s5) {
        return s1 + s2 + s3 + s4 + s5;
    }

    // Function to calculate percentage
    static double percentage(int total) {
        return (total * 100.0) / 500;
    }

    // Function to display grade
    static void grade(double percentage) {
        if (percentage >= 90 && percentage <= 100) {
            System.out.println("Grade: A");
        } else if (percentage >= 80 && percentage < 90) {
            System.out.println("Grade: B");
        } else if (percentage >= 70 && percentage < 80) {
            System.out.println("Grade: C");
        } else if (percentage >= 60 && percentage < 70) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F");
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.println("Enter marks of 5 subjects:");
        int sub1 = sc.nextInt();
        int sub2 = sc.nextInt();
        int sub3 = sc.nextInt();
        int sub4 = sc.nextInt();
        int sub5 = sc.nextInt();

        int total = totalMarks(sub1, sub2, sub3, sub4, sub5);
        double per = percentage(total);
    }
}
```

```
System.out.println("Total Marks: " + total);
System.out.println("Out Of: 500");
System.out.println("Percentage: " + per);
grade(per);
sc.close();
}
}
```

Function	Return Type	Input	Logic
totalMarks()	int	s1,s2,s3,s4,s5	return s1+s2+s3+s4+s5
percentage()	double	total (int)	return (total * 100.0) / 500
grade()	void	percentage (double)	if-else chain → prints grade

§ D

Number Logic

Armstrong numbers 1–1000 and prime numbers 1–100 — nested loops and Math.pow

PROGRAM 07

ArmstrongNumbers1To1000.java

Print ALL Armstrong numbers from 1 to 1000 — generalized n-digit algorithm

■ CONCEPT — Generalized Armstrong — Three Loops, Two Passes per Number

Day 2 checked a single hardcoded number (153). This program uses an outer **for loop** to test every number from 1 to 1000. For each candidate, **Pass 1** counts its digits (using a while loop + /10), and **Pass 2** extracts each digit, raises it to that count using **Math.pow(digit, count)**, and accumulates. The final sum is cast to int implicitly — if it equals the original number, it's Armstrong.

■ ArmstrongNumbers1To1000.java

```
package Day3;

public class ArmstrongNumbers1To1000 {
    public static void main(String[] args) {
        System.out.println("Armstrong numbers from 1 to 1000:");
        for (int num = 1; num <= 1000; num++) {
            int temp = num;
            int count = 0;
            int sum = 0;

            // Pass 1: Count digits
            while (temp != 0) {
                count++;
                temp = temp / 10;
            }

            temp = num; // Reset temp for Pass 2
            // Pass 2: Calculate Armstrong sum
            while (temp != 0) {
                int digit = temp % 10;
                sum += Math.pow(digit, count);
                temp = temp / 10;
            }

            // Check Armstrong
            if (sum == num) {
                System.out.print(num + " ");
            }
        }
    }
}
```

■ Output

```
Armstrong numbers from 1 to 1000:
1 2 3 4 5 6 7 8 9 153 370 371 407
```

Algorithm Trace — num = 153

Pass	temp	Operation	Result
Pass 1	153 → 15 → 1 → 0	/10 until 0	count = 3
Pass 2	153	digit=3, sum+=3^3=27	sum=27
Pass 2	15	digit=5, sum+=5^3=125	sum=152
Pass 2	1	digit=1, sum+=1^3=1	sum=153
Check	—	sum(153) == num(153)	■ Armstrong!

Why Single-Digit Numbers (1–9) are Always Armstrong

Number	Digit count	Calculation	Armstrong?
1	1	1^1 = 1 == 1	■ Yes
5	1	5^1 = 5 == 5	■ Yes
9	1	9^1 = 9 == 9	■ Yes
10	2	1^2 + 0^2 = 1 ≠ 10	■ No
153	3	1^3+5^3+3^3=153	■ Yes
370	3	3^3+7^3+0^3=370	■ Yes
371	3	3^3+7^3+1^3=371	■ Yes
407	3	4^3+0^3+7^3=407	■ Yes

■ KEY vs Day 2 Version

Day 2's `ArmstrongNumber.java` checked **ONE** hardcoded number (153) and the digit count was always 3. This version handles **any n-digit number** because count is re-computed per number using a while loop. That's what makes **`Math.pow(digit, count)`** generalize correctly to 1-digit, 3-digit, and beyond.

PROGRAM 08

PrimeinRange.java

Print all prime numbers from 1 to 100 — nested loop + flag variable

■ CONCEPT — Prime Range — Outer Loop Iterates Candidates, Inner Counts Divisors

This upgrades the Day 2 single-number prime check into a **range finder**. The outer loop iterates every candidate from 2 to 100. For each, an inner loop counts divisors using a **flag** variable — the same pattern from Day 2's `PrimeNumber.java`. If `flag == 2` (exactly divisors 1 and itself), the number is prime and gets printed.

■ PrimeinRange.java

```
package Day2;

public class PrimeinRange {

    public static void main(String[] args) {

        System.out.println("Prime numbers from 1 to 100:");

        for (int num = 2; num <= 100; num++) {

            int flag = 0;

            for (int i = 1; i <= num; i++) {

                if (num % i == 0) {

                    flag++;

                }

            }

            if (flag == 2) {

                System.out.print(num + " ");

            }

        }

    }

}
```

```

    }
}

if (flag == 2) {
    System.out.print(num + " ");
}
}
}
}
}

```

■ Output

Prime numbers from 1 to 100:

2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97

Algorithm Trace — num = 7

i (inner loop)	7 % i == 0?	flag after
1	7 % 1 = 0 ■ Yes	1
2	7 % 2 = 1 ■ No	1
3	7 % 3 = 1 ■ No	1
4	7 % 4 = 3 ■ No	1
5	7 % 5 = 2 ■ No	1
6	7 % 6 = 1 ■ No	1
7	7 % 7 = 0 ■ Yes	2
Exit	flag == 2 → 7 is Prime	■ Printed

num = 6 (not prime)	i	6 % i == 0?	flag
	1	6%1=0 Yes	1
	2	6%2=0 Yes	2
	3	6%3=0 Yes	3
	4	6%4=2 No	3
	5	6%5=1 No	3
	6	6%6=0 Yes	4
	Exit	flag==4 ≠ 2	Not printed

■ OPTIMIZATION TIP

The current algorithm runs i from 1 to num — $O(n)$ divisor checks per candidate. For competitive programming, only check i from 2 to $\sqrt{num}$: **for (int i=2; i*i <= num; i++)** — if no divisor found, it's prime. This reduces complexity from $O(n^2)$ to $O(n\sqrt{n})$ for the full range.

§ E

Master Quick Reference
All 8 programs · Concept cheat sheet · Key Java concepts recap

All 8 Programs — Complete Reference

#	Program	Core Concept	Key Line / Formula	Output
01	DoWhileGuessNumber.java	do-while – at least one execution	while (guess != secretNumber)	Correct Guess!
02	Game.java	do-while + guess counter	count++ inside loop	N guesses taken
03	PassByValueDemo.java	Pass by value – copy, not reference	return num; (only way out)	10 → 11 → 11
04	ModularApp.java	Static methods + while+switch	while menu = Deposit(balance, amount)	Full bank menu
05	MarksPercentageGradeCal.java	Array + for loop input/display	total += marks[i]	Grade: B
06	GradeFunction.java	3 functions: total, %, grade	static int totalMarks(...)	Modular output
07	ArmstrongNumbers1To1000.java	2-pass: count digits, sum	pow += Math.pow(digit, count)	1..9 153 370 371 407
08	PrimeinRange.java	Nested loops + flag counter	flag++ → if flag==2: prime	2 3 5 7 ... 97

Key Java Concepts Used — Theory Recap

Concept	What It Is	Used In
do-while	Executes body first, checks condition after — guaranteed ≥1 run	DoWhileGuessNumber, Game
Pass by value	Primitive arguments are COPIES — original unchanged inside method	PassByValueDemo, ModularApp
static method	Method callable without creating an object — belongs to the class	ModularApp, GradeFunction, PassByValueDemo
return	Sends a value back from a method to the caller	Deposit, Withdraw, totalMarks, percentage
void	Return type when a method does not return any value	showbalance, FD, grade
Array int[]	Fixed-size sequence of same-type elements, indexed 0..n-1	MarksPercentageGradeCal
Math.pow(a,b)	Returns a^b as double — in java.lang, no import needed	ArmstrongNumbers1To1000
Flag variable	Counter tracking a condition across loop iterations	PrimeinRange
Nested loops	Outer loop = candidates, inner loop = divisor counting	PrimeinRange, ArmstrongNumbers1To1000
while + switch	Menu-driven pattern: while keeps loop alive, switch routes options	ModularApp
total * 100.0	Forces double arithmetic to avoid integer truncation in %	MarksPercentageGradeCal, GradeFunction

do-while Pattern Cheat Sheet

Use Case	Why do-while?	Condition Placed
Number guessing game	User must guess at least once before result can be checked	while (guess != secret)
Menu-driven app	Menu must display at least once before user can choose Exit	while (option != exit)
Input validation	Must read input once before checking if it is valid	while (!isValid(input))
Retry logic	Must attempt once before deciding to retry	while (!success)